

ユニット 2・第2週

# システムコールと低水準入出力

📅 第2週・日程は未定    📄 演習 TD1（解答 + 生成AIチャレンジ）    📺 アニメーション 4 本

演習 TD1・解答付き

## read と write で読み書きする

システムプログラミングの基礎：プログラムがカーネルに入る方法、バイナリで読み書きする方法、そしてすべての結果を確認すべき理由。

### ☑ 実習レポート

この実習は、授業のテンプレートを使った短い LaTeX レポートで提出します。各演習の説明、要点となるコード、端末で実行したことを書きます。LaTeX の作成には生成 AI を使えます。利用を申告し、すべての結果を確認してください。テンプレート、例、Prism のガイド →

### 演習問題

- |   |                                          |   |                          |
|---|------------------------------------------|---|--------------------------|
| 1 | システムコールの実装                               | 5 | <code>read</code> でコピーする |
| 2 | <code>write</code> で作る <code>echo</code> | 6 | 整数を読み戻す                  |
| 3 | 整数をバイナリで書く                               | 7 | バイナリの構造体とパディング           |
| 4 | <code>perror</code> はシステムコール？            |   |                          |

TD1 ・ 1

● 解答済み・実際の出力

## システムコールの実装

課題。システムコールにはプロセッサの特別な命令（x86-64 では `syscall`、ARM では `svc`、RISC-V では `ecall`）が必要です。 `objdump -d` で C ライブラリを逆アセンブルし、プログラムがどの呼び出しを望んでいるかをカーネルにどう伝えているかを見つけてください。次に、あなたの `echo` が行う呼び出しを `strace` で観察しましょう。

解答



#### 実際の出力

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A4 -E '^[0-9a-f]+ <getppid@>'
0000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

#### 実際の出力

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A6 -E '^[0-9a-f]+ <__write_nocancel@>'
0000000000129060 <__write_nocancel@GLIBC_PRIVATE>:
    129060:    endbr64
    129064:    mov     $0x1,%eax
    129069:    syscall
    12906b:    cmp     $0xffffffffffffffff000,%rax
    129071:    ja      129078 <__write_nocancel@GLIBC_PRIVATE+0x18>
    129073:    ret
```

呼び出し番号はレジスタ `eax` に (x86-64 では `110 = getpid`、`1 = write`)、引数は `rdi`、`rsi`、`rdx` ... に入ります。 `syscall` 命令は CPU をカーネルモードに切り替えてカーネルの入口へジャンプし、カーネルは `eax` をシステムコール表の添字として使います。戻ると結果は `rax` にあります: `-4095` から `-1` の値はエラーを意味し、`glibc` のラッパーがそれを `-1 + errno` に変えます ( `cmp $0xffffffffffffffff000,%rax` がそれを行っています)。

#### 実際の出力

```
$ strace -e trace=write ./echo hola mundo > /dev/null
write(1, "hola", 4)                = 4
write(1, " ", 1)                  = 1
write(1, "mundo", 5)              = 5
write(1, "\n", 1)                 = 1
+++ exited with 0 +++
```

`write` は 4 回: 単語ごとに 1 回、空白に 1 回、改行に 1 回。それぞれがカーネルへの往復です。ユニット 4 では、標準ライブラリが書き込みをバッファにまとめて節約するしくみを見ます。

TD1 · 2

● 解答済み・テスト済み

**write(2) で作る echo**

課題。 `write(2)` だけを使って引数を標準出力にコピーする `echo` プログラムを書いてください。  
`write` のエラーは `exit_if` で確認すること。 `./echo blabla < . 1>&0` で試してみましょう。

解答

C tds/td1/echo.c

```
/*
 * TD1 - exercise 3: echo with write(2).
 * Copies its arguments to standard output, separated by spaces, followed by
 * a newline.
 * Try the error path with: ./echo blabla < . 1>&0 (stdout becomes a
 * directory: write fails)
 */
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    for (int i = 1; i < argc; i++) {
        size_t len = strlen(argv[i]);
        ssize_t rc = write(STDOUT_FILENO, argv[i], len);
        exit_if(rc == -1, "write");
        /* write may write less than asked (pipe or socket full): it is not
         * an error,
         * but this program must then say so. write_all() in libutils loops
         * instead. */
        exit_if((size_t)rc != len, "write: partial write");

        if (i < argc - 1)
            exit_if(write(STDOUT_FILENO, " ", 1) == -1, "write space");
    }
    exit_if(write(STDOUT_FILENO, "\n", 1) == -1, "write newline");
    return EXIT_SUCCESS;
}
```

#### 実際の出力

```
$ ./echo hola mundo
hola mundo
```

#### 実際の出力

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

`< .` はカレントディレクトリを標準入力（ディスクリプタ 0）として開き、`1>&0` は出力（1）をディスクリプタ 0 の複製にします。読み取り専用で開いたディレクトリには書き込めません：`write` は `errno = EBADF` で `-1` を返し、`exit_if` がそれを報告します。

TD1 ・ 3

● 解答済み・テスト済み

## 整数をバイナリで書く

課題。`write_number` を書いてください：値 5 の `long` のメモリ内容を標準出力に書き、続けて 1 回目の呼び出しが書いたバイト数を書きます。端末には何が見えますか？出力を `od` で見てみましょう。

解答



C tds/td1/write\_number.c

```
/*
 * TD1 - exercise 4: write the memory content of a long, then the number of
 * bytes written.
 * The output is binary, not text: look at it with ./write_number | od -A d
 * -t x1
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    long value = 5;
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);    /* 8 bytes on
x86-64 */
    exit_if(written == -1, "write value");

    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

#### 実際の出力

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
00000016
```

端末には読める文字は何も現れません：バイト `05 00 00 00 00 00 00 00` は表示可能な文字ではないからです。`od` が真実を示します：リトルエンディアン順（最下位バイトが先。x86-64 プロセッサが整数を格納する順序）の `long` の 8 バイトと、1 回目の `write` が返した 8 を表す 8 バイト。`write` は何もテキストに変換せず、バイトをコピーするだけです。

TD1 ・ 4

● 解答済み

## perror はシステムコール？

解答



いいえ。pperror はマニュアルのセクション 3 にあり、errno を読み、エラーの文言を探して標準エラーに書く C ライブラリの関数です...内部では write システムコールを使っています。  
strace ./echo blabla < . 1>&0 で確かめられます：  
write(2, "write: Bad file descriptor\n", ...) が現れます。

TD1 ・ 5

● 解答済み・テスト済み

## read(2) で入力を出力にコピーする

課題。read を書いてください：標準入力を read でサイズ BUFFER\_SIZE の配列に読み込み、write で出力に書きます。入力が閉じられる (Ctrl + D) まで続けます。cat として、ファイルのコピーで、そしてディレクトリを入力にして試しましょう。

解答



C tds/td1/read.c

```

/*
 * TD1 - exercise 6: copy standard input to standard output with
 * read(2)/write(2).
 * Stops at end of file (Ctrl-D in a terminal). Usages:
 * ./read          ./read < file          ./read < in > out
 * ./read < . (error)
 */
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#ifndef BUFFER_SIZE
#define BUFFER_SIZE 4096
#endif

int main(void)
{
    char buffer[BUFFER_SIZE];

    for (;;) {
        ssize_t n = read(STDIN_FILENO, buffer, BUFFER_SIZE);
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "read");
        if (n == 0) /* end of file: the writer closed its
side */
            return EXIT_SUCCESS;

        /* write can be partial: loop until the n bytes are out (see
write_all in libutils). */
        ssize_t done = 0;
        while (done < n) {
            ssize_t rc = write(STDOUT_FILENO, buffer + done, (size_t)(n -
done));
            exit_if(rc == -1, "write");
            done += rc;
        }
    }
}

```

#### 実際の出力

```
$ ./read < input.txt
first line
second line
```

#### 実際の出力

```
$ ./read < binary.bin > copy.bin && cmp binary.bin copy.bin && echo identical
identical
```

#### 実際の出力

```
$ ./read < .
read: Is a directory
[exit status 1]
```

#### 💡 正しいプログラムと「ほとんどいつも」動くプログラムを分ける2つの点

- `read` はファイル終端で `0`、エラーで `-1`、それ以外は最大 `BUFFER_SIZE` バイトを返します：それより少ないこともあります（パイプ、端末、シグナルが読み込みを短くする）。
- `write` も要求より少なく書くことがあります：だから内側のループは `n` バイトを出し切るまで繰り返します。何も書く前にシグナルで中断されると `EINTR` で失敗し、再試行すれば十分です。

TD1 ・ 6

● 解答済み・テスト済み

### `write_number` の整数を読む

課題。 `write_number` が出力したものを読み、 `printf` で表示するプログラムを書いてください。

解答



C tds/td1/read\_number.c

```

/*
 * TD1 - exercise 7: read what write_number produces and print it as text.
 * ./write_number | ./read_number      →  value = 5, bytes written = 8
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

/* read() may return fewer bytes than asked (e.g. from a pipe): loop until
`len` bytes. */
static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    long value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("value = %ld, bytes written = %zd\n", value, written);
    return EXIT_SUCCESS;
}

```

#### 実際の出力

```

$ ./write_number | ./read_number
value = 5, bytes written = 8

```

`read_exact` は要求したバイト数がちょうど得られるまで `read` を繰り返します：  
パイプ越しでは 16 バイトが 2 つに分かれて届くことがあるからです。これがうまくいくのは  
両方のプログラムが同じマシンで動き、`long` のサイズとバイト順序を共有しているからです。  
異なるマシンの間では形式を取り決める必要があります（たとえば `htonl` による  
ネットワークバイト順、ユニット 8）。

TD1 · 7

● 解答済み・テスト済み

## バイナリの構造体とパディング

課題。前の 2 つのプログラムを、`utils.h` で宣言した

`struct nopad { char c1; long l; char c2; }`（値は `'a'`、5、`'b'`）用の `write_struct` と  
`read_struct` に作り替えてください。構造体の大きさは？ `-03` で変わる？どうすれば小さく  
できる？

### 解答

```
C tds/td1/write_struct.c

/*
 * TD1 - exercise 8: write a structure in binary. struct nopad is declared in
 * utils.h
 * so that write_struct and read_struct agree on its layout.
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    struct nopad value = { .c1 = 'a', .l = 5, .c2 = 'b' };
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);
    exit_if(written == -1, "write value");
    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

C tds/td1/read\_struct.c

```

/*
 * TD1 - exercise 8: read the structure written by write_struct.
 * ./write_struct | ./read_struct      → { c1 = 'a', l = 5, c2 = 'b' },
bytes written = 24
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    struct nopad value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("{ c1 = '%c', l = %ld, c2 = '%c' }, bytes written = %zd\n",
           value.c1, value.l, value.c2, written);
    return EXIT_SUCCESS;
}

```

#### 実際の出力

```

$ ./write_struct | ./read_struct
{ c1 = 'a', l = 5, c2 = 'b' }, bytes written = 24

```

C tds/td1/struct\_sizes.c

```

/*
 * TD1 - exercise 8 (answers): size and padding of struct nopad, and two ways
 * to shrink it.
 * The compiler aligns `long` on 8 bytes: 7 padding bytes after c1 and 7
 * after c2 (arrays of
 * the structure must keep every `l` aligned). -O3 does not change the
 * layout: it is part of
 * the ABI, not an optimisation.
 */
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

struct reordered {          /* biggest member first: only 6 padding bytes at
the end */
    long l;
    char c1;
    char c2;
};

struct __attribute__((packed)) packed { /* no padding at all (slower,
unaligned access) */
    char c1;
    long l;
    char c2;
};

int main(void)
{
    printf("struct nopad      : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct nopad),
        offsetof(struct nopad, c1), offsetof(struct nopad, l),
        offsetof(struct nopad, c2));
    printf("struct reordered : size %2zu  offsets l=%zu c1=%zu c2=%zu\n",
sizeof(struct reordered),
        offsetof(struct reordered, l), offsetof(struct reordered, c1),
        offsetof(struct reordered, c2));
    printf("struct packed    : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct packed),
        offsetof(struct packed, c1), offsetof(struct packed, l),
        offsetof(struct packed, c2));
}

```

```
return EXIT_SUCCESS;
}
```

#### 実際の出力

```
$ ./struct_sizes
struct nopad      : size 24  offsets c1=0 l=8 c2=16
struct reordered  : size 16  offsets l=0 c1=8 c2=9
struct packed     : size 10  offsets c1=0 l=1 c2=9
```

フィールドの合計は10 バイトなのに、構造体は **24 バイト** になります：コンパイラは `long` を 8 バイト境界にそろえ、`c1` の後に 7 バイト、末尾に 7 バイトのパディングを加えます（構造体の配列でも各 `l` がそろうように）。`-03` では何も変わりません：メモリ上の配置は最適化ではなく *ABI* の一部だからです。小さくするには：フィールドを大きい順に並べる（16 バイト）か、`__attribute__((packed))` で詰める（10 バイト。ただし境界のそろわないアクセスは遅く、移植性もない）。

#### ⚠ 引き継いだ版からの修正

`read_struct.c` はコンパイルできませんでした：構造体が `utils.h` になく、存在しないフィールド `c` を使っていました。

## チェックリスト

### 生成AIチャレンジ・自分のシステムコールを測る `cat`



**目標。** `read.c` を拡張してバッファサイズを引数で受け取るようにし、各サイズで `read` と `write` を何回呼ぶかを `strace -c` と実行時間で測り、その曲線を説明してください。

**最初のプロンプト。** 好きなツール (Claude、ChatGPT、Gemini、Copilot、Codex...) で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

#### Prompt

read/write と固定サイズのバッファで標準入力を標準出力にコピーする C プログラムがあります [read.c を貼る]。次のことを手伝ってください：(1) malloc を使ってバッファサイズ (1 バイト～1 MiB) を引数で受け取る、(2) サイズ 1、16、256、4096、65536、1048576 のそれぞれで 50 MB のファイルをコピーし、`/usr/bin/time -f %e` で時間を測り、`strace -c` で呼び出しを数える bash スクリプトを書く、(3) 最初は時間が大きく下がり、その後頭打ちになる理由を説明する。私が数値を見る前に結論を言わないでください。

#### 結果の確かめ方

- どのサイズでもコピーしたファイルは同一 (`cmp`)。
- `read` の回数は  $\approx$  ファイルサイズ / バッファサイズ (さらに 0 を返す最後の読み込みが 1 回)。
- 各システムコールの固定コスト (モード切り替え) を自分の言葉で説明できる。

**AI の利用を申告しましょう：**使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。